



Universidad De Las Fuerzas Armadas - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

FUNDAMENTOS DE PROGRAMACIÓN

TIPOS DE DATOS ABSTRACTOS (TDA)

Docente:

Ing. César Iván Chilibingua Mendoza

Estudiante:

Tiban Camalle Andy Mathew

27 de julio de 2026

1. Introducción

La organización y estructuración de la información es un aspecto fundamental en la resolución de problemas mediante software. Para lograrlo, se utilizan los Tipos de Datos Abstractos (TDA), los cuales permiten definir datos y las operaciones que pueden realizarse sobre ellos, sin necesidad de conocer cómo están implementados internamente. En C++, los TDA pueden desarrollarse mediante **struct** o **class**, facilitando la creación de modelos de datos personalizados y manteniendo separada la interfaz del usuario de la implementación del programa.

OBJETIVOS

General

Investigar y conceptualizar los Tipos de Datos Abstractos (TDA), su declaración, definición de variables, acceso, almacenamiento y lectura de información mediante el lenguaje C++.

Específicos

- Diferenciar entre un TDA y una estructura de datos convencional mediante sus ventajas en el desarrollo de software.
- Analizar el proceso de declaración, definición y asignación en memoria de variables de un TDA en C++.
- Demostrar de manera práctica, mediante código C++, el acceso, almacenamiento y lectura de datos utilizando abstracción y encapsulamiento.

Justificación y Alcance

Este informe responde a la necesidad de fortalecer los fundamentos de programación estructurada y modular. Comprender los TDA en este nivel constituye el puente conceptual necesario para la posterior transición hacia la Programación Orientada a Objetos (POO). El alcance comprende el análisis teórico de los TDA y ejemplos prácticos implementados en C++ utilizando estructuras/clases y constructores/métodos básicos.

2. Marco Teórico

¿Qué es un Tipo de Dato Abstracto (TDA)?

Un TDA es un modelo matemático compuesto por una colección de datos y un conjunto de operaciones definidas sobre ellos. Su característica principal es la abstracción de datos: quien utiliza el TDA sabe qué hace el tipo de dato, pero ignora cómo está implementado internamente.

Diferencia entre TDA y Estructura de Datos

La diferencia principal es que un **TDA** (Tipo de Dato Abstracto) define las operaciones que se pueden realizar con los datos, sin importar cómo están implementados. En cambio, una **estructura de datos** es la forma en que esos datos se almacenan y organizan en la memoria de la computadora.

Ventajas de utilizar TDA

- **Modularidad:** Facilita la división del programa en módulos independientes.
- **Encapsulamiento y Reutilización:** Oculta los detalles internos, protegiendo los datos contra accesos no autorizados y permitiendo reutilizar el código fácilmente.
- **Mantenibilidad:** Si se cambia la implementación interna de un TDA, el resto del programa no se ve afectado mientras la interfaz se mantenga.

Relación entre los TDA y la Programación Orientada a Objetos (POO)

Los TDA son la base teórica de la POO. En C++, la POO formaliza los TDA mediante el concepto de Clase (donde los datos son *atributos* y las operaciones son *métodos*), agregando principios como herencia y polimorfismo.

3. Desarrollo/Resultados

DECLARACIÓN DE UN TDA

Declarar un TDA significa dar a conocer al compilador la existencia del nuevo tipo de dato, indicando sus miembros (variables) y sus operaciones asociadas, sin reservar memoria aún.

Su **finalidad** es establecer el contrato o interfaz con el que interactuará el programa.

Los elementos de una declaración son:

- **Nombre:** El identificador único que tendrá el TDA.
- **Firmas de funciones:** Los nombres de los métodos, operaciones o funciones asociadas.
- **Parámetros:** El tipo de datos que reciben dichas funciones.
- **Retorno:** El tipo de dato que devuelve cada operación.

La diferencia entre declaración y definición radica en que la **declaración** solo anuncia la existencia y la firma del TDA (el "qué hace"). No reserva espacio en memoria ni escribe la lógica interna. La **definición** implementa el código real de las funciones y especifica la estructura de datos interna (el "cómo lo hace"). Se encarga de reservar el espacio físico en memoria.

EJEMPLO EN PSEUDOCÓDIGO

```
EJEMPLO_TDA.psc

1  //TDA CuentaBancaria
2
3  Operaciones:
4      CrearCuenta(nombre, saldoInicial)
5      Depositar(monto)
6      Retirar(monto)
7      ConsultarSaldo()
```

Como se definió un TDA, no se especifica como se almacena los datos, solo se define lo que se puede hacer con ellos.

EJEMPLO EN C++

```

1  class Date {
2      private:
3          int day;
4          int month;
5          int year;
6
7      public:
8          void ingresarFecha();
9          void mostrarFecha();
10         bool fechaValida();
11     };

```

El usuario de la clase no necesita saber cómo se almacenan los datos. Solo necesita conocer las operaciones disponibles.

Definición de Variables y Memoria

Significado: Definir una variable de un TDA implica reservar un bloque físico de memoria de tamaño suficiente para albergar los miembros definidos por el TDA.

¿Qué ocurre en memoria? El compilador calcula la suma del tamaño de cada uno de sus componentes más el padding de alineación e identifica una dirección base en el Stack (o Heap si se usa memoria dinámica).

Diferencia entre Declarar una Clase y Crear un Objeto: Declarar la clase define el "molde" o plantilla (ocupa 0 bytes de espacio para datos). Crear un objeto asigna memoria física real en tiempo de ejecución basada en dicho molde.

```

1  struct Fecha {
2      int dia;
3      int mes;
4      int anio;
5  };
6
7  Fecha cumpleaños;

```

Cumpleaños:

dia= ¿?

Mes=¿?

Anio=¿?

Acceso a la Información

Significado: Consultar o modificar el contenido interno almacenado en un TDA.

Mecanismos: Se efectúa a través de sus métodos públicos o mediante acceso directo si es un struct público.

Operadores en C++:

- . : Operador punto (acceso desde una variable/objeto directo).
- > : Operador flecha (acceso a través de un puntero al objeto).

EJEMPLO EN C++

```

1  #include <iostream>
2  using namespace std;
3
4  struct Estudiante {
5      string nombre;
6      int edad;
7      float promedio;
8  };
9
10 int main(){
11
12     Estudiante alumno;
13
14     //guardar informacion
15     alumno.nombre = "Andy";
16     alumno.edad = 19;
17     alumno.promedio = 12.81;
18
19     //acceder a la informacion
20     cout << alumno.nombre << endl;
21     cout << alumno.edad << endl;
22     cout << alumno.promedio << endl;
23
24
25     return 0;
26 }

```

Almacenamiento de Información

- **Significado:** Escribir o actualizar valores dentro de los miembros de un TDA asegurando la integridad de los datos.
- **Métodos:** Constructores, métodos Setters (modificadores) o asignación directa.
- **Ventajas de encapsular el almacenamiento:** Evita estados inconsistentes (por ejemplo, asignar una edad negativa o una nota superior al límite) mediante validaciones internas.

EJEMPLO

```

1  struct Fecha {
2      int dia;
3      int mes;
4      int anio;
5  };
6
7  int main() {
8
9      Fecha nacimiento;
10
11     nacimiento.dia = 15;
12     nacimiento.mes = 8;
13     nacimiento.anio = 2006;
14
15     cout << nacimiento.dia << "/"
16         << nacimiento.mes << "/"
17         << nacimiento.anio;
18     return 0;
19 }

```

Lectura de Información

Leer: Leer información significa obtener o consultar los datos que ya fueron almacenados, sin modificarlos.

Recuperar: Significa buscar un dato que estaba almacenado y traerlo para usarlo.

Funciones utilizadas: Flujos de entrada (std::cin, getline) para lectura, y métodos Getters para recuperación, dependiendo de donde se encuentren los datos.

LEER INFORMACION	RECUPERAR INFORMACIÓN
<pre> 1 struct Estudiante { 2 string nombre; 3 int edad; 4 }; 5 6 Estudiante alumno; 7 8 alumno.nombre = "Andy"; 9 alumno.edad = 19; </pre>	<pre> 1 if (alumnos[i].nombre == nombreBuscado) { 2 3 cout << alumnos[i].nombre << endl; 4 cout << alumnos[i].edad << endl; 5 cout << alumnos[i].promedio << endl; 6 } 7 </pre>

4. Análisis/Discusión

Durante la práctica se analizó el funcionamiento de los Tipos de Datos Abstractos (TDA) utilizando struct en C++. Se comprobó que es posible crear tipos de datos definidos por el usuario para agrupar información relacionada, como el nombre, la edad y el promedio de un estudiante. Además, se observó cómo almacenar información asignando valores a cada uno de los miembros de la estructura y cómo acceder a ellos mediante el operador punto (.). También se realizaron operaciones de lectura y recuperación de información, verificando que los datos almacenados pueden consultarse y utilizarse posteriormente dentro del programa. Esta práctica permitió comprender la importancia de organizar la información en una sola estructura, facilitando el manejo de los datos y mejorando la claridad del código.

5. Conclusiones

En esta práctica se comprendió la importancia de los **Tipos de Datos Abstractos (TDA)** para organizar y manejar información relacionada dentro de un programa. Mediante el uso de struct en C++, fue posible crear tipos de datos definidos por el usuario, almacenar información, acceder a sus miembros y realizar operaciones de lectura y recuperación de datos. Además, se identificó la diferencia entre el concepto de un TDA y su implementación mediante una estructura de datos, entendiendo que un TDA define qué información y operaciones existen, mientras que la estructura de datos establece cómo se almacena esa información en memoria. Estos conocimientos constituyen una base importante para el desarrollo de programas más organizados, claros y fáciles de mantener.

6. Recomendaciones

- Continuar practicando la creación de Tipos de **Datos Abstractos (TDA)** utilizando **struct** y **class** para comprender mejor su aplicación en C++.

- Utilizar nombres descriptivos para las estructuras, variables y miembros, con el fin de mejorar la legibilidad y el mantenimiento del código.
- Validar los datos ingresados por el usuario antes de almacenarlos para evitar errores durante la ejecución del programa.
- Aplicar los TDA en problemas más complejos que involucren varios objetos, con el propósito de fortalecer la organización y reutilización de la información.

7. Referencias

- Joyanes Aguilar, L. (2008). Programación en C++: Algoritmos, estructuras de datos y objetos (2.^a ed.). McGraw-Hill.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- Weiss, M. A. (2014). Estructura de datos en C++ (4.^a ed.). Pearson Educación.
- Ojeda, L. R. *Tda Programacion Orientado a Objetos en Turbo*. Univ. Nacional de Colombia.
- Molinero, L. D. H. (2023). Tecnología de la programación: Grado en Matemáticas. Programación Conjunta de Estudios Oficiales Grado en Matemáticas y Grado en Física. Tecnología de la programación: Grado en Matemáticas. Programación Conjunta de Estudios Oficiales Grado en Matemáticas y Grado en Física.
- Hernández Molinero, L. D. (2023). Tecnología de la programación. Editum. Ediciones de la Universidad de Murcia. <https://doi.org/10.6018/editum.3013>